

Chapter 1

The Spatio-Temporal Traffic Forecaster

1.1 Introduction

The digital twin of Chapter ?? consumes an *offered load* and reports what a UPF would do at that load. It does not say where the load comes from. This chapter builds the component that does: a spatio-temporal forecaster that predicts near-future traffic for each of the K geographic service areas into which a city’s base stations are grouped.

As in the rest of this thesis, it is worth being precise about scope, because the system is deliberately decomposed into independent parts:

The forecaster is not a calibrator. It emits a *dimensionless* per-cluster load $\hat{d}$ (the sum of member-station loads, each normalised to $[0, 1]$). The mapping to physical Gbps is the scenario constant α owned by the twin (Chapter ??, Eq. 1.1); it is applied *after* forecasting and never enters training. One trained forecaster therefore serves every capacity scenario.

The forecaster is not a controller. It predicts load; it does not choose a UPF configuration. Controllers consume the forecast (and its error bar) but are out of scope here (Chapter ??).

This separation mirrors the repository structure: the forecaster is a standalone project whose only coupling to the twin is a fixed file contract (Section 1.5).

The contribution of this chapter is not a new forecasting architecture but a documented spatial pipeline and an evidence-based answer to three questions a system designer must settle before wiring a forecaster to the twin. The pipeline (Sections 1.2–1.3) turns raw per-tile measurements into a signal on a graph of K contiguous service areas: Voronoi attribution of tiles to base stations, an adjacency graph over the stations, and a geographic clustering whose structure is characterised in its own right. The questions are the following. First, *which forecaster* to use: Section 1.7 benchmarks six models from a linear baseline to established spatio-temporal graph networks under a common protocol, and finds the field tight, with a simple, cheap model losing little to the most sophisticated. Second, at *what spatial granularity* one should forecast: is it better to aggregate base stations into service areas and forecast the cluster signal directly (*cluster-first*), or to forecast every base station and sum? Section 1.11 answers this with a controlled ablation and finds the cluster-first route dramatically better — aggregation *creates* predictability by denoising sparse per-station traffic. Third, whether the *geographic* form of that aggregation matters for forecasting: Section 1.12 replaces the service areas by k -means and by random groupings of the same sizes, and shows that the predictability comes from pooling stations rather than from their contiguity — geography is a deployment requirement whose cost in forecast error can be measured. The headline finding is that *what* is forecast matters far more than *which model* forecasts it.

Place in the thesis. Within this thesis’s theme of *augmented observability* for the 5/6G data plane, the forecaster extends the observability envelope along its *temporal* axis. The profiling campaign made per-function power and QoS observable at the present instant; the forecaster pushes the horizon forward, making the *near future* of per-cluster load observable one control step ahead. A controller that could see only the present would have to react to congestion after it arrived; forecasting is what lets the downstream twin and controllers act before the load does. Its accuracy is therefore not a private metric but part of the observability contract: the twin sizes its anti-flapping band directly from the forecast error, so forecast quality propagates into control behaviour (Section 1.5).

The chapter proceeds as follows. Section 1.2 describes the data and builds the spatial substrate — Voronoi cells and the base-station graph — and characterises the traffic on it. Section 1.3 clusters the base stations into service areas and derives the forecasting graph. Section 1.4 introduces the forecasting models, Section 1.5 states the output contract consumed by

the twin, and Section 1.6 defines the evaluation methodology and the skill metric. Section 1.7 benchmarks the models and selects one; Section 1.8 dissects its error by cluster, lead time and load regime; Sections 1.9–1.10 characterise it across cluster counts and traffic types. Sections 1.11 and 1.12 then test the two spatial design decisions themselves — forecasting at the cluster level, and clustering geographically. Section 1.13 summarises.

1.2 Data and Spatial Substrate

Before any forecasting model is involved, three steps turn raw operator measurements into a signal on a graph: the measurements are normalised, attributed to base stations, and the base stations are connected into an adjacency graph. This section documents those steps and characterises the result, because the properties established here — how sparse a single station is, how fast correlation decays in space — are what the design decisions of the following sections rest on.

1.2.1 Dataset and normalisation

Dataset. The forecaster is driven by NetMob 2023, the same source the twin’s NetMob adapter targets: per-tile downlink traffic for Lyon over 77 days (2019-03-16 to 2019-05-31) at 15-minute resolution (96 slots/day; $T = 7388$ slots, one hour being absent at the daylight-saving change), for the services Netflix, YouTube, and DailyMotion. The spatial unit is a $100\text{ m} \times 100\text{ m}$ tile; the Lyon footprint comprises 54 013 tiles covering 539 km^2 . The raw values are privacy-preserving aggregate indicators with *no physical unit*; this is the structural reason the pipeline is normalised and the twin needs α .

Normalisation. Tile traffic is aggregated to base stations and divided by a single global maximum computed over all services,

$$\hat{d} = \frac{\text{bs_tile_sum}}{\text{global_max}}, \quad \text{global_max} = 1.482 \times 10^8, \quad (1.1)$$

so that $\hat{d} \in [0, 1]$ for every base station. A *single* normaliser (rather than one per service) is used precisely so that the cross-service ratios are preserved and cluster signals can be formed by direct summation: the mean city-wide load of Netflix (4.39) is close to that of YouTube (4.28) and $\approx 4.7\times$ that of DailyMotion (0.94), differences the normalisation retains. Note that only

the *per-station* load is bounded by one; a cluster signal is a sum over member stations and routinely exceeds it (Section 1.3).

Data quality. The trace contains one collection outage, on 12 May 2019, during which the recorded load is near zero from 01:00 to 18:00. The series is used as recorded. Under the chronological split of Section 1.6 the outage falls entirely inside the validation window: it touches neither the training data nor any reported test metric, and it perturbs the early-stopping criterion identically for every model.

1.2.2 From tiles to base stations: Voronoi attribution

NetMob reports traffic per tile, whereas the network is operated per site. Tiles are attributed to sites with a Voronoi tessellation built from the 965 distinct gNodeB locations inside the Lyon footprint (ANFR Cartoradio; co-located antennas are merged). The tessellation is computed in a local metric projection and each cell is clipped to the city footprint; a tile is assigned to the cell that contains its centroid, and a station’s load is the sum over its tiles. The Voronoi cell is the standard zero-information approximation of a coverage area: in the absence of antenna azimuths, powers and propagation data, it assigns every location to its nearest site.

Figure 1.1(a) shows the resulting cells. Their size varies by nearly three orders of magnitude, following the site density: the median cell covers 0.34 km^2 (34 tiles), the mean 0.56 km^2 , and the largest peripheral cell 7.5 km^2 (Figure 1.2(a)). Traffic is concentrated where the cells are smallest: the load density of the central cells exceeds that of the periphery by up to two orders of magnitude.

1.2.3 The base-station graph

Two base stations are adjacent when their clipped Voronoi cells share a border of at least 1 m; the edge is weighted by the inverse of the distance between the two sites. The resulting graph (Figure 1.1(b)) has 965 nodes and 2772 undirected edges and is connected. As expected of a planar tessellation its degree distribution is narrow (mean 5.7, median 6, range 1–11; Figure 1.2(b)), while edge lengths are right-skewed (median 0.60 km, 90th percentile 1.48 km, maximum 4.1 km; Figure 1.2(c)), again reflecting the dense centre and sparse periphery. This graph plays two roles: it is the contiguity constraint for the clustering of Section 1.3, and it is the fine graph from which every coarse forecasting graph is derived.

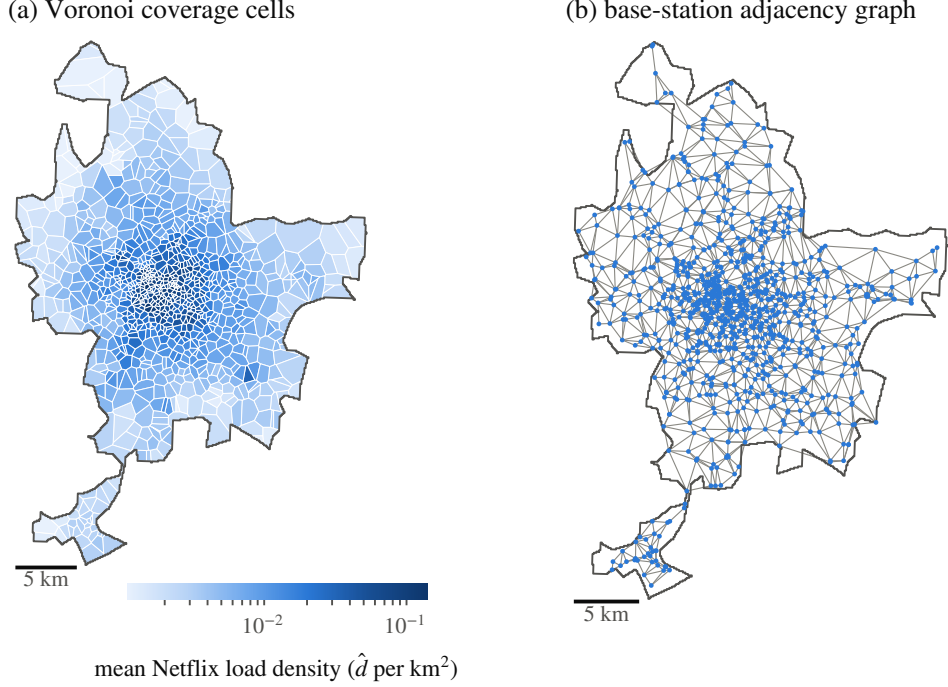


Figure 1.1: Spatial substrate. (a) Voronoi coverage cells of the 965 base stations, clipped to the Lyon footprint and shaded by mean Netflix load density (logarithmic scale). (b) The base-station adjacency graph: two stations are connected when their cells share a border (2 772 edges).

1.2.4 Traffic characterisation

Four properties of the signal, summarised in Figure 1.3, shape everything that follows.

The city-level signal is strongly diurnal. All three services share a night-time trough near 05:00 and differ in their daytime shape (Figure 1.3(a)): Netflix peaks sharply in the late evening at roughly twice its mean, YouTube plateaus through the afternoon and evening, and DailyMotion shows a pronounced weekday midday peak. Weekend profiles rise later in the morning.

A single base station is sparse. For Netflix, 88% of all station-slots carry $\hat{d} < 0.01$, and the median station is below that level 93% of the time; for DailyMotion the figure is 99% (Figure 1.3(b)). Per-station traffic is thus a sequence of isolated bursts on a near-zero floor.

Aggregation restores temporal structure. The autocorrelation of a typical

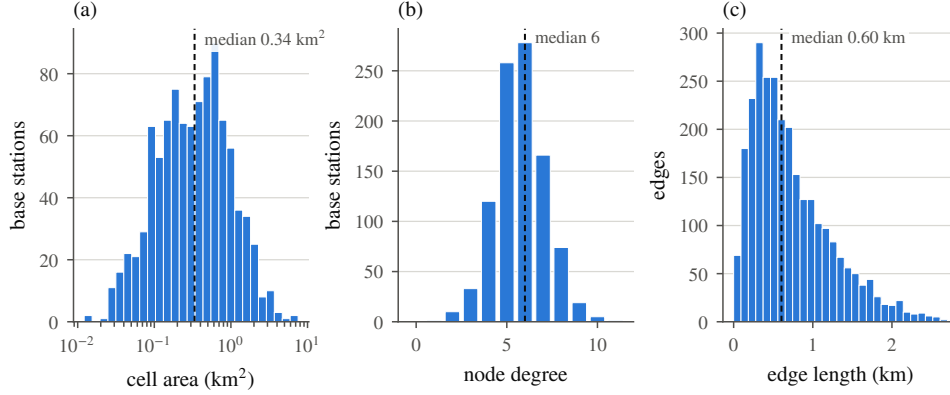


Figure 1.2: Statistics of the spatial substrate: (a) Voronoi cell area (logarithmic axis), (b) node degree, and (c) edge length of the base-station adjacency graph. Dashed lines mark medians.

station drops to 0.58 after one 15-minute slot and reaches only 0.24 at the 24-hour lag; for the ten-cluster signals of Section 1.3 the corresponding values are 0.80 and 0.60, and for the city aggregate 0.96 and 0.86 (Figure 1.3(c)). The daily rhythm that makes traffic forecastable is barely present at a single station and emerges only as stations are summed — the observation that the ablation of Section 1.11 turns into a controlled result.

Correlation is local. The median correlation between two stations is 0.74 when they are adjacent, 0.44 at two hops and 0.31 at three, and reaches the city-wide background level of ≈ 0.23 by about six hops (Figure 1.3(d)). Spatial dependence therefore exists, is short-ranged, and is aligned with the Voronoi adjacency — which motivates both grouping *neighbouring* stations and giving the forecaster a graph.

1.3 Service-Area Clustering

With 965 base stations, neither per-station control nor — as Section 1.11 will show — per-station forecasting is viable. The stations are therefore grouped into K service areas, each of which is the catchment of one edge UPF site and one node of the forecasting graph.

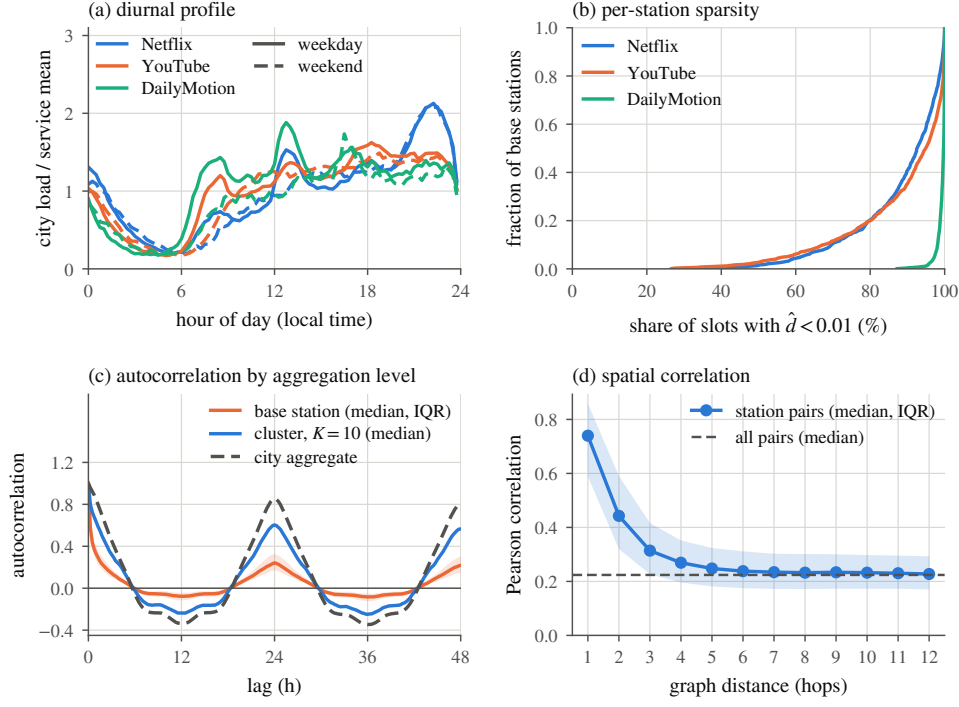


Figure 1.3: Traffic characterisation. (a) Mean diurnal profile of the city-wide load per service, relative to the service mean (solid: weekdays, dashed: weekends). (b) Empirical distribution over base stations of the share of slots with $\hat{d} < 0.01$. (c) Autocorrelation of the Netflix signal at three aggregation levels. (d) Pearson correlation between the Netflix series of two base stations as a function of their hop distance in the adjacency graph (median and inter-quartile range).

1.3.1 Requirements and method

A partition into service areas must satisfy three requirements that follow from the deployment rather than from forecasting. (i) *Contiguity*: a service area is the set of cells anchored to one user plane function, so it must be a single connected region. (ii) *Compactness*: the distance from a site to its UPF bounds the transport latency, so areas should be geographically tight. (iii) *Service independence*: the partition describes infrastructure and must not change with the traffic type being forecast.

These requirements are met by SKATER [5], a divisive regionalisation method based on pruning a spanning tree of the adjacency graph. A mini-

mum spanning tree of the base-station graph is built, with edges weighted by inter-site distance; removing an edge from a tree splits it into two connected subtrees, so any set of $K - 1$ edge removals yields K contiguous regions by construction. The edges are removed greedily: at each step the method cuts the edge that most reduces the within-region sum of squared deviations of the site coordinates from their region centroid,

$$\text{SSD} = \sum_{k=1}^K \sum_{i \in \mathcal{C}_k} \|\mathbf{x}_i - \bar{\mathbf{x}}_k\|^2, \quad (1.2)$$

subject to both resulting regions containing at least three base stations. The procedure is deterministic and uses coordinates only, so requirement (iii) holds trivially. Because each step splits exactly one existing region, the partitions are *nested* across K : the service areas at $K+1$ refine those at K , which makes results at different K directly comparable. One implementation detail should be noted: distances and deviations are evaluated in geographic coordinates (degrees) rather than in a metric projection, which at Lyon’s latitude under-weights east–west separations by a factor of about 0.70 relative to north–south ones.

Two alternatives were considered. Plain k -means on site coordinates yields compact areas but ignores the adjacency graph, so contiguity is not guaranteed; it is kept as a comparison in Section 1.12. An attributed spectral clustering that mixed geographic affinity with load-magnitude and temporal-shape features was explored in an earlier iteration and abandoned: the traffic-feature terms produced spatially scattered areas and singletons, and made the partition depend on the service, violating requirements (i) and (iii).

1.3.2 Resulting service areas

Figure 1.4 shows the service areas for three values of K , and Table 1.1 summarises their structure over the full range studied. Every cluster is connected at every K , and compactness improves steadily, the mean distance from a station to its cluster centroid falling from 4.1 km at $K=4$ to 0.7 km at $K=100$.

Two properties of the partition deserve emphasis, because they propagate to the twin and the controllers.

The service areas are compact but not balanced. SKATER minimises geographic dispersion, not load or membership imbalance, and Lyon’s site density is strongly peaked at the centre. At $K=10$ (Figure 1.5) the central

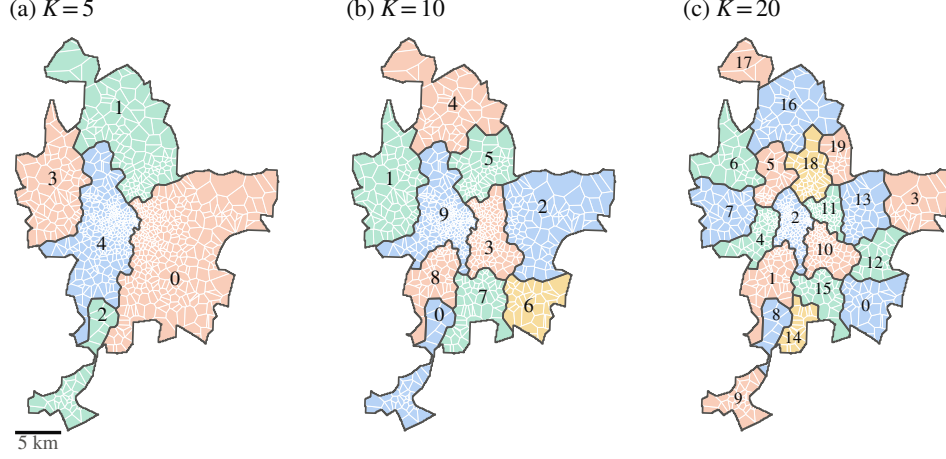


Figure 1.4: SKATER service areas for $K = 5, 10$ and 20 . Thin white lines are Voronoi cells, dark lines service-area boundaries; numbers are cluster identifiers. Colours only separate neighbouring areas. The partitions are nested: each area at a larger K lies inside one area at a smaller K .

cluster contains 297 of the 965 stations and carries 36% of the load, with a mean of $\hat{d} = 1.57$ and a 99th percentile of 4.3, while the two smallest peripheral clusters (36 and 42 stations) average 0.14 and 0.10; the coefficient of variation of cluster load is close to one at every K . This imbalance is a feature of the city, not an artefact to be removed: it is what makes the downstream configuration decision non-trivial, since clusters differ by an order of magnitude in the capacity they require. It also means that pooled error metrics are dominated by the few heavy clusters, which is why Section 1.8 reports errors per cluster as well.

Geographic clusters are more coherent than chance, increasingly so as they shrink. The mean correlation between two stations of the same cluster rises from 0.31 at $K=4$ to 0.61 at $K=100$, against 0.24 for size-matched random groupings at every K (Table 1.1). This is the short-range correlation of Figure 1.3(d) seen through the partition: the tighter the area, the more its members move together.

1.3.3 Coarsening to the forecasting graph

Each cluster becomes one graph node. Two clusters are connected when any member base station of one is Voronoi-adjacent to any member of the other, and the edge weight is the mean geographic affinity (a Gaussian kernel

Table 1.1: Structure of the SKATER partition as a function of K (Netflix load). Radius is the mean distance of a station to its cluster centroid; load CV is the coefficient of variation of mean cluster load; “largest” columns give the share of all base stations and of all load held by the largest cluster; $\bar{\rho}_{\text{in}}$ is the mean pairwise correlation between stations of the same cluster (0.24 for a random partition with the same cluster sizes, at every K).

K	stations per cluster	largest (%)		radius	load	coarse	$\bar{\rho}_{\text{in}}$
	min / median / max	stations	load	(km)	CV	edges	
4	42 / 244 / 436	45	49	4.06	0.75	5	0.31
6	42 / 117 / 357	37	43	3.64	0.86	9	0.30
8	42 / 88 / 357	37	43	3.06	1.02	14	0.32
10	36 / 70 / 297	31	36	2.73	0.99	18	0.33
15	10 / 55 / 191	20	26	2.19	1.04	28	0.37
20	6 / 39 / 191	20	26	1.84	1.12	40	0.40
30	6 / 24 / 101	10	12	1.50	0.88	67	0.44
50	6 / 17 / 70	7	8	1.11	0.89	121	0.50
100	3 / 8 / 34	4	5	0.70	0.92	258	0.61

on site coordinates) over those cross-border station pairs; self-loops of unit weight are added. The node signal at each slot is the *sum* of the member stations’ $\hat{d}$,

$$\hat{d}_k(t) = \sum_{i \in \mathcal{C}_k} \hat{d}_i(t), \quad (1.3)$$

so traffic is conserved exactly: the cluster signals sum to the city total at every slot, which the pipeline verifies numerically. The result is a K -node graph and a $(K \times T)$ signal matrix. The coarse graph is sparse — 18 edges at $K=10$, a mean degree of 3.6 — and remains so as K grows (Table 1.1). The complete pipeline is summarised in Figure 1.6.

1.4 Forecasting Models

Every model predicts, for all nodes simultaneously, the next $H = 4$ slots (1h) from the previous $L = 96$ slots (24h): input (B, L, N) of normalised loads (one feature per node) plus, for graph models, the static graph (edge_index, edge_attr); output (B, H, N) , with $N = K$ clusters at the cluster level. To answer “which forecaster?” without favouring any paradigm, the benchmark of Section 1.7 spans, under one training protocol:

Naive floors. *Persistence* (repeat the last value) and *seasonal-naive* (the same slot 24h earlier) — the references the skill metric is measured

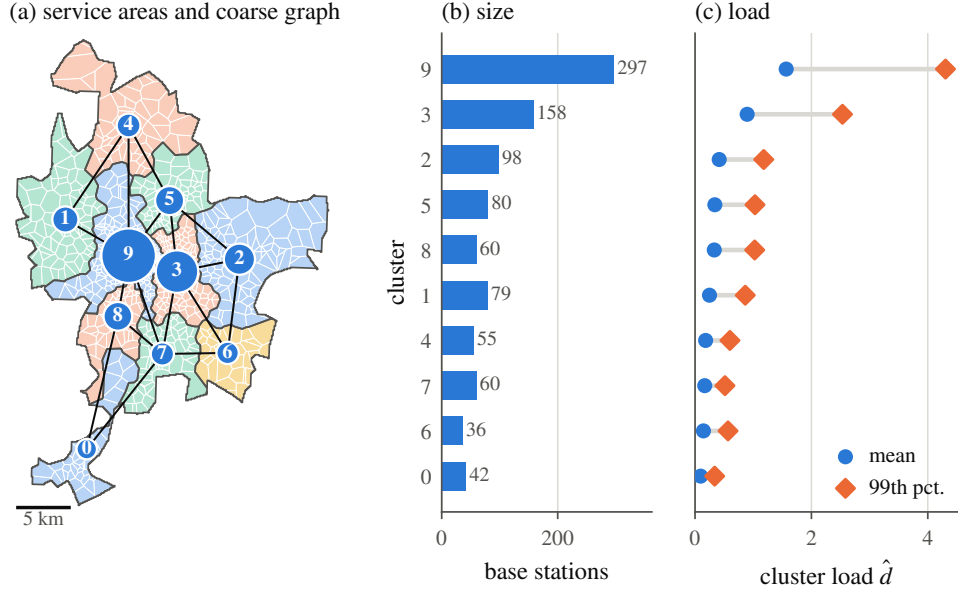


Figure 1.5: The $K=10$ partition used as the reference configuration. (a) Service areas with the coarse forecasting graph overlaid; node area is proportional to mean cluster load, and an edge joins two clusters that share a border. (b) Number of base stations and (c) mean and 99th-percentile Netflix load per cluster, ordered by decreasing mean load.

against.

DLinear [4] — a channel-independent trend/seasonal decomposition with two linear maps; the “is anything deep needed?” baseline.

GRU (no graph) — a per-node gated recurrent encoder; the purely temporal model, and the natural ablation of the graph (any graph model’s edge over it is the value of spatial information).

DCRNN [1] — diffusion-convolutional recurrence; spatial mixing fused into a GRU cell over the given graph.

Graph WaveNet [2] — dilated temporal convolutions with a learnable adjacency.

AGCRN [3] — a node-adaptive GRU over an adjacency the model *learns* from data, ignoring the supplied graph (a test of whether a better graph than Voronoi exists).

[DRAW.IO PLACEHOLDER]

Forecasting pipeline / data flow. Left-to-right block diagram. (1) *Raw NetMob*: 100 m tile grid over Lyon, 15-min slots (icon: heatmap of tiles). (2) *Voronoi assignment*: tiles $\rightarrow$ 965 gNodeBs (icon: Voronoi cells over a map, base-station dots). (3) *Normalise*: $\hat{d} = \text{bs_sum} / \text{global_max} \in [0, 1]$. (4) *SKATER clustering*: 965 base stations $\rightarrow K$ contiguous clusters (icon: map recoloured into K regions). (5) *Coarsen*: each cluster = one node; edges where clusters share a Voronoi border; node signal = sum of member $\hat{d}$. (6) *STGNN*: K -node graph + 24 h history $\rightarrow$ 1 h-ahead per-cluster $\hat{d}$. Show the output arrow leaving to “Digital Twin (Ch. ??)” to emphasise the hand-off. Mark clearly that everything left of the STGNN is one-time preprocessing and the cluster structure is shared across services.

Figure 1.6: The cluster-first forecasting pipeline: raw tiles are aggregated to base stations, clustered geographically into K nodes, and forecast at the cluster level.

GAT-GRU (in-house) — a per-node GRU temporal encoder followed by a two-layer graph-attention spatial mixer (4 heads) and a linear head; included as a reasonable in-house spatio-temporal baseline, not as a proposed method.

The graph models share the same coarse K -node graph; the GRU and DLinear ignore it. All are *compact reimplementations on a common harness*: results reflect each architecture’s inductive bias under identical data, budget, and protocol rather than its best-tuned reference performance — the appropriate footing for a controlled comparison.

1.5 Outputs and the Twin Data Contract

The forecaster does not call the twin; it writes files the twin reads. For a chosen (service, K) it exports seven artifacts into a flat directory consumed by the twin’s loader: test-window predictions and targets (`predictions_test.npy`, `targets_test.npy`), the full cluster signal (`cluster_series.npy`), the cluster $\rightarrow$ base-station map and assignment table, the base-station coordinates, and the evaluation summary (`forecast_eval_summary.json`). The arrays are in normalised units of $\hat{d}$; the twin applies α . Critically, the forecaster’s reported mean absolute error is part of the contract: the twin derives its anti-flapping hysteresis band

[DRAW.IO PLACEHOLDER]

Example spatio-temporal architecture (the in-house GAT-GRU).

Input tensor $(B, L=96, N)$ on the left. Lane 1 “Temporal”: per-node GRU (hidden 64) over the 96-step series $\rightarrow$ node embeddings $(B, N, 64)$. Lane 2 “Spatial”: 2-layer GAT (4 heads) over the N -node graph, drawn as nodes with weighted edges and little attention-head fans. “Output head”: linear $\rightarrow (B, H=4, N)$. Annotate the static graph inputs (edge_index, edge_attr) feeding the GAT. Keep colours consistent with Figure 1.6. This figure is illustrative of the STGNN family; the other graph models differ in how spatial and temporal mixing are composed.

Figure 1.7: An example spatio-temporal forecaster (the in-house GAT-GRU): a per-node GRU temporal encoder feeding a graph-attention spatial mixer and a linear forecast head.

as $2 \cdot \text{MAE}_{\text{forecast}}$ (Chapter ??, Eq. 1.6), so forecast accuracy propagates directly into control behaviour.

1.6 Evaluation Methodology

Protocol. The data is split chronologically (70/15/15 train/val/test; no shuffling) to prevent leakage. Models train with Adam, an L1 (MAE) objective, cosine learning-rate decay, and early stopping on validation MAE (patience 15). Unless stated, results are averaged over three seeds.

Metrics. Forecasts are scored in $\hat{d}$ units with MAE, RMSE, and WAPE,

$$\text{WAPE} = 100 \cdot \frac{\sum |y - \hat{y}|}{\sum |y|}, \quad (1.4)$$

which is used instead of MAPE because traffic is sparse and MAPE explodes on near-zero denominators.

Skill, not raw error. Absolute error is misleading across graph sizes: as K grows, each cluster aggregates fewer base stations, so its signal is burstier and *intrinsically* harder in relative terms even though its absolute magnitude shrinks. The model-independent quantity is therefore *skill* over a baseline,

$$\text{skill} = 1 - \frac{\text{WAPE}_{\text{model}}}{\text{WAPE}_{\text{baseline}}}, \quad (1.5)$$

Table 1.2: Model comparison at $K=10$ and $K=50$ (Netflix, seed-averaged over 3 seeds). Skill is over persistence (higher is better); WAPE in %. Best skill per column in bold; training time ranges over the two graph sizes and seeds. All models are compact reimplementations under a common protocol.

model	params	$K=10$		$K=50$		train (s)
		WAPE	skill	WAPE	skill	
persistence	—	29.5	0.000	44.4	0.000	—
DLinear	0.8k	23.6	0.199	36.2	0.185	4–7
GRU (no graph)	13k	22.6	0.234	34.8	0.217	15–60
Graph WaveNet	78k	22.0	0.253	34.2	0.229	50–170
DCRNN	75k	21.9	0.258	33.9	0.236	650–1040
AGCRN	252k	22.3	0.243	34.1	0.233	1670–1790
GAT-GRU (in-house)	60k	23.0	0.219	35.0	0.211	120–680

reported against persistence (last value) and seasonal-naive (same slot 24 h earlier); a historical-mean baseline is also computed. Skill isolates what the model contributes from how hard the target intrinsically is.

1.7 Which Forecaster? A Model Benchmark

The system forecasts one signal per service area (Section 1.3), so the first operative question is: which model best forecasts at the cluster level? Table 1.2 and Figure 1.8 benchmark six learned forecasters (Section 1.4) under one protocol at $K \in \{10, 50\}$ on Netflix.

Three findings stand out.

First, spatial information helps — modestly. Every model that uses the given graph or learns one (Graph WaveNet, DCRNN, AGCRN) beats the graph-free GRU at both sizes (0.253/0.258/0.243 vs. 0.234 at $K=10$): a real, repeatable $\sim 3\%$ WAPE reduction. But the gain is small and does not grow with K , and *learning* the adjacency (AGCRN) does not beat the given Voronoi graph — so the ceiling is the weak residual spatial signal at cluster level, not the graph’s quality: once stations are pooled into service areas, little is left for the inter-cluster graph to add.

Second, the field is tight and simple models lose little. From DLinear (0.199) to the best model (0.258) the entire spread is ~ 0.06 skill, and the no-graph GRU (0.234) is within ~ 0.02 of the best. Model choice barely moves the needle.

Third, accuracy-cost favours the simple. DCRNN leads but is among the slowest; Graph WaveNet matches it to within 0.005 skill at roughly a tenth

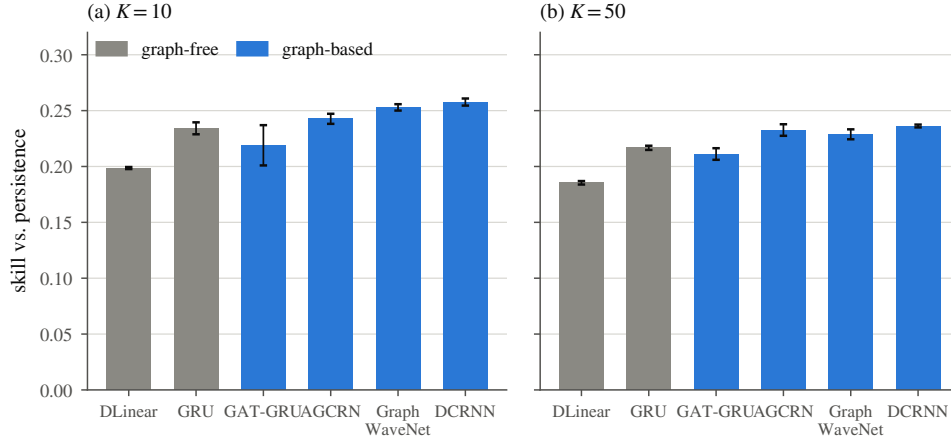


Figure 1.8: Skill over persistence of the six learned forecasters at (a) $K=10$ and (b) $K=50$ (Netflix; mean over three seeds, error bars ± 1 std). Grey bars are graph-free models, blue bars models that use or learn a graph. The whole field spans only ~ 0.06 skill: the graph models lead the graph-free GRU by a small margin, and even a linear model is close behind.

of the cost (its dilated convolutions parallelise where DCRNN’s recurrence does not); AGCRN is the costliest (252k parameters, ~ 28 min/run) for no gain; and the in-house GAT-GRU does not lead. We therefore adopt **Graph WaveNet** as the deployed forecaster (best accuracy-cost); a plain **GRU** would be a defensible simpler choice. The rest of the chapter characterises Graph WaveNet, and Section 1.11 shows the one decision the system is *not* insensitive to — forecasting granularity — dwarfs every model difference here.

1.8 Anatomy of the Forecast Error

The benchmark compares models through one pooled number. A controller, however, consumes one forecast per cluster, at a given lead time, and is most sensitive to errors when load is high. This section therefore opens up the pooled figure for the selected model at the reference configuration (Graph WaveNet, $K=10$, Netflix; three seeds, independently retrained with test predictions retained: pooled WAPE 22.1%, skill 0.249, consistent with Table 1.2). Figure 1.9 shows representative forecasts and Figure 1.10 the three breakdowns.

The advantage is uniform across clusters. Raw WAPE varies widely

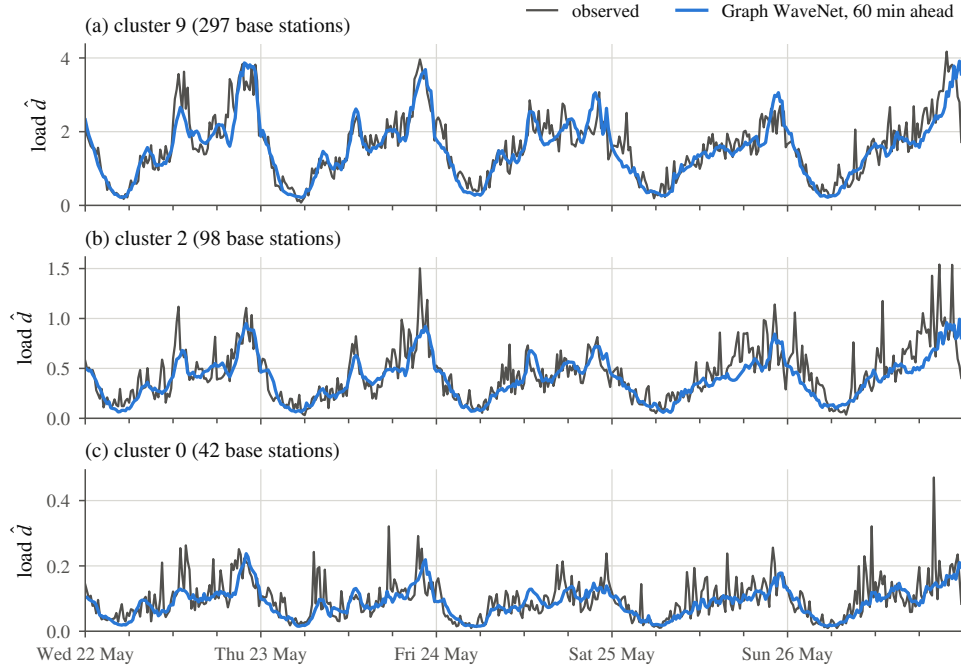


Figure 1.9: Observed and forecast load, 60 minutes ahead, over five days of the test window for the largest, a mid-sized and the smallest-load cluster of the $K=10$ partition (Graph WaveNet, one seed). The forecast tracks the diurnal envelope at every scale; short spikes, increasingly prominent as the cluster shrinks, are not anticipated.

between clusters, from 16% for the central cluster to 40% for the smallest-membership one, mirroring the size effect of Section 1.9. Skill does not: it lies between 0.22 and 0.29 for all ten clusters (Figure 1.10(a)), with no cluster on which the model fails to improve on persistence and no systematic dependence on cluster load. The pooled figure is therefore representative, not an average over good and bad areas, despite the load imbalance documented in Section 1.3.

The advantage grows with lead time. Persistence degrades quickly as the horizon lengthens (WAPE 23.5% at 15 minutes, 34.5% at 60), whereas the model degrades slowly (20.1% to 23.7%), so skill rises from 0.14 at 15 minutes to 0.31 at 60 (Figure 1.10(b)). At 60 minutes persistence is already worse than the seasonal-naïve forecast. The model is thus most valuable exactly where a controller needs it: a configuration change has to be decided ahead

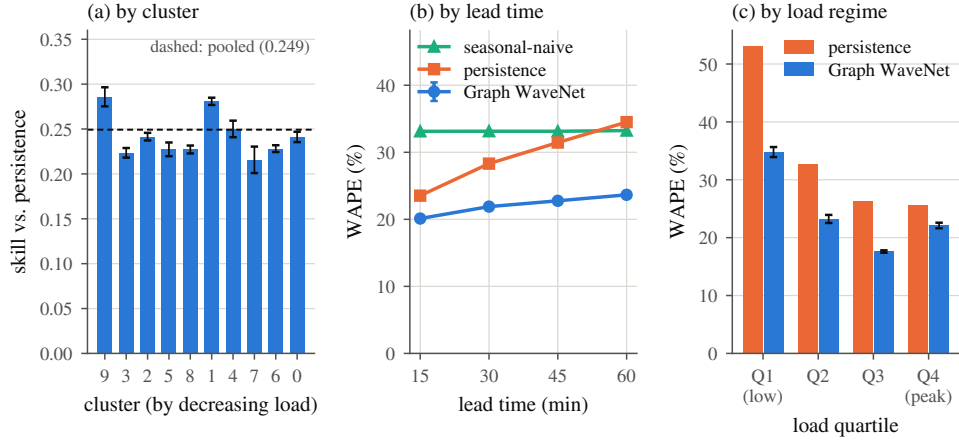


Figure 1.10: Breakdown of the Graph WaveNet test error at $K=10$ (Netflix; mean over three seeds, error bars ± 1 std). (a) Skill over persistence per cluster, clusters ordered by decreasing mean load. (b) WAPE by lead time, against both naive baselines. (c) WAPE by quartile of the realised load, computed per cluster.

of the load it will serve, so it is the far end of the horizon, not the next slot, that the decision depends on.

Peaks are under-forecast. Relative error is highest in the lowest load quartile (34.8%), where absolute loads are small, and lowest in the third (17.6%); in the top quartile it is 22.1%, against 25.6% for persistence, so the margin over persistence narrows from 0.29–0.35 in the lower three quartiles to 0.14 at peak (Figure 1.10(c)). More important than the size of the error is its sign. The forecast is biased upward at low load (+11% in the bottom quartile) and downward at high load (−17% in the top quartile, −35% over the top 1% of slots): it regresses toward the diurnal envelope and misses the short spikes visible in Figure 1.9. This is the expected behaviour of a model trained with an absolute-error objective, which targets the conditional median of a right-skewed quantity. It matters downstream, because under-forecasting a peak is the costly direction for a capacity decision. The twin’s hysteresis band, sized from the forecast MAE (Section 1.5), absorbs part of it; a quantile or asymmetric training objective would address it at the source and is the most direct improvement left open by this chapter.

Table 1.3: Graph WaveNet forecast quality vs. cluster count K (Netflix, seed-averaged). Raw WAPE rises and MAE falls with K — both intrinsic to shrinking clusters — while *skill* over the baselines stays flat, the signature of a model that scales.

K	test WAPE	test MAE	skill vs. persist.	skill vs. seasonal
4	16.4	0.192	0.266	0.333
6	18.9	0.147	0.250	0.328
8	20.5	0.120	0.248	0.329
10	22.0	0.103	0.253	0.337
15	24.8	0.078	0.239	0.333
20	26.7	0.062	0.238	0.334
30	30.4	0.048	0.231	0.335
50	34.2	0.032	0.229	0.338
100	38.8	0.018	0.232	0.347

1.9 Effect of Cluster Count K

The number of MEC clusters K is a deployment parameter — how finely the operator partitions the city. How does the chosen forecaster (Graph WaveNet) behave as K grows from a handful of clusters to one hundred? Table 1.3 sweeps $K \in \{4, \dots, 100\}$ (Netflix, 3 seeds).

Across a $25\times$ change in K , Graph WaveNet holds a near-constant ~ 0.23 – 0.27 skill over persistence and ~ 0.33 over seasonal-naive, even as raw WAPE climbs from 16.4 to 38.8 and MAE falls from 0.192 to 0.018. The rising WAPE and falling MAE are two views of one intrinsic effect — smaller clusters aggregate fewer stations, so their signal is burstier in *relative* terms but smaller in *absolute* terms — while the flat skill curve shows the model keeps the same advantage regardless of granularity (this is precisely why skill, not raw WAPE, is the headline metric). For the operator the practical consequence is that K can be chosen on deployment grounds — number of MEC sites, controller tractability — without paying a forecasting penalty.

1.10 Generalization Across Traffic Types

Because SKATER clustering is geographic (Section 1.3), the *same* cluster structure can be re-used for a different service, changing only the signal on the nodes. This is a clean external-validity test: does Graph WaveNet’s performance hold for traffic with a different spatial and temporal profile? Ta-

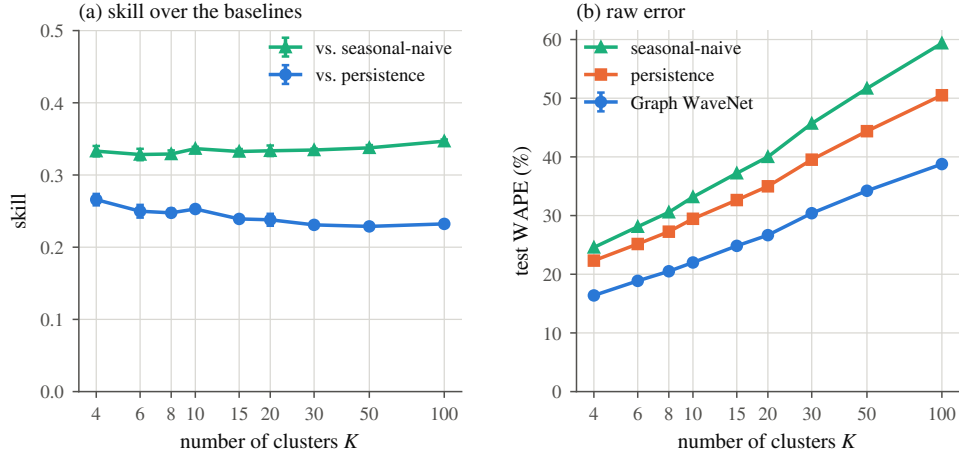


Figure 1.11: Graph WaveNet from 4 to 100 clusters (Netflix; mean over three seeds, error bars ± 1 std). (a) Skill over both naive baselines is preserved across the whole range. (b) Raw test WAPE grows with K for the model and for both baselines alike: the rise reflects the intrinsically harder small-cluster signal, not a loss of model quality.

Table 1.4: Graph WaveNet skill over persistence by traffic type (seed-averaged). Positive for every service; the margin tracks predictability — largest on smooth aggregate traffic, smallest on sparse DailyMotion.

service	$K=8$	$K=10$	$K=15$
total (aggregate)	0.305	0.287	0.281
Netflix	0.248	0.253	0.239
DailyMotion (sparse)	0.140	0.160	0.153

ble 1.4 reports skill over persistence for the aggregate of all services (**total**) and the sparse **DailyMotion**, alongside Netflix, at $K \in \{8, 10, 15\}$ (3 seeds, frozen architecture).

Skill is positive for every service and every K , so the result is not Netflix-specific: it holds for aggregate and sparse traffic on the same geographic graph. The margin behaves as predictability would suggest — widest on the smooth **total** signal (~ 0.29), narrower on Netflix, and smallest on the heavily sparse DailyMotion (~ 0.15 vs. persistence). DailyMotion is the instructive case: its persistence-skill is low because its traffic is bursty, yet its *seasonal*-skill is the highest of all (~ 0.37) — the sparse signal is very repetitive day-to-day, so the model beats persistence only modestly but beats

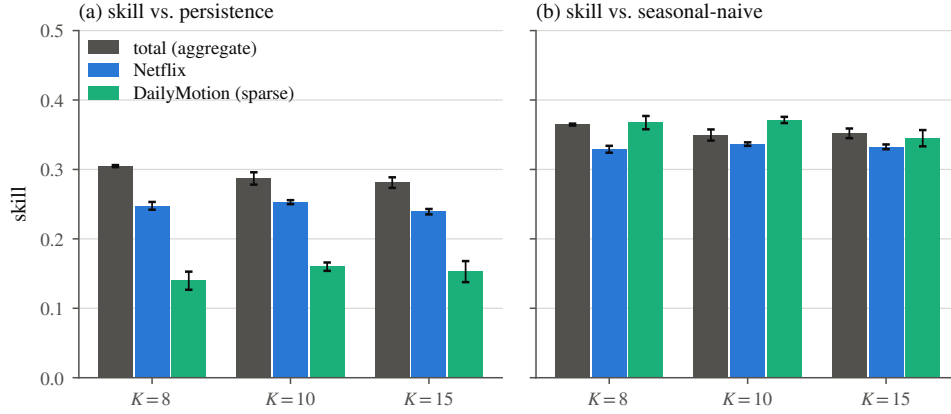


Figure 1.12: Graph WaveNet skill by traffic type on the same geographic clusters (mean over three seeds, error bars ± 1 std). (a) Against persistence the margin scales with the predictability of each service. (b) Against seasonal-naive, DailyMotion’s skill rises to ~ 0.37 , above Netflix, despite its sparsity.

seasonal-naive strongly. The honest scope is robustness *across traffic types within Lyon*: the services share city, base stations, and period, so this is not evidence of cross-city transfer.

1.11 Cluster-First vs. Per-Base-Station: Why Aggregate?

The preceding sections chose a model (Graph WaveNet) and showed it forecasts cluster load robustly across graph sizes and traffic types. All of it rests on one prior decision: that we forecast at the *cluster* level at all. The MEC deployment already dictates clusters — but is aggregating first also a *forecasting* advantage, or merely a convenience we would pay for in accuracy? This section tests that decision head-on.

Section 1.9 cannot answer it — it scores each K against its own (changing) target. A controlled test instead fixes the delivered target (the $K=10$ cluster signal) and compares two routes to it:

Route A (cluster-first). Aggregate to $K=10$ clusters; forecast the cluster signals directly.

Route B (per-base-station). Forecast all 965 base stations; *sum* the pre-

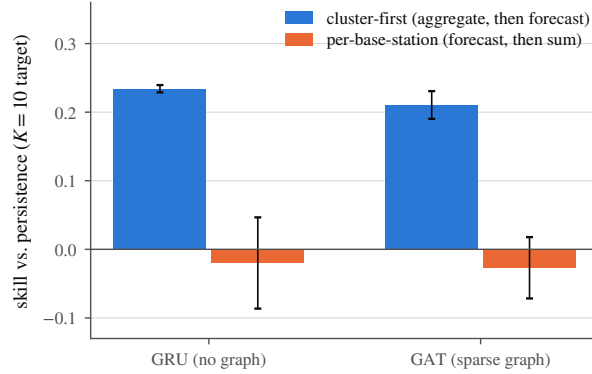


Figure 1.13: Forecasting the aggregated cluster signal directly (cluster-first) versus forecasting 965 base stations and summing (per-base-station), scored on the same $K=10$ target. The per-station route collapses to persistence-level skill (≈ 0); cluster-first reaches 0.21–0.23 (mean over three seeds, error bars ± 1 std).

dictionaries into the same 10 clusters.

Both are scored on the identical $K=10$ target — the 965-station series sums exactly to the cluster series (verified to floating-point error) — so the comparison isolates *when* aggregation happens. The route is run with the graph-free GRU and a sparse graph-attention STGNN; the dense-adjacency models (Graph WaveNet, DCRNN, AGCRN) do not even scale to 965 nodes, which is itself telling.

The result (Figure 1.13) is decisive: forecasting at the base-station level and summing yields a cluster forecast *no better than persistence* (skill -0.02 to -0.03), whereas forecasting the cluster signal directly reaches 0.21–0.23 — consistent across both models and all three seeds. The mechanism is *denoising by aggregation*: per-station traffic is extremely sparse (for Netflix, 88% of station-slots are below 0.01; Section 1.2.4), so 965 such series are nearly unpredictable individually, and a sum of persistence-level forecasts is itself persistence-level. The *sum*, by contrast, is a smooth, strongly diurnal signal that is easy to forecast.

This reframes the role of aggregation. Pooling stations before forecasting is not a loss of information to be tolerated for the sake of tractability: it is what makes the problem forecastable at all, and no model choice comes close to the gap seen here. **Cluster-first is a forecasting advantage, not merely a tractability convenience.** (A far larger per-station model

might recover some skill, but the gap — 0.22 versus ≈ 0 — is too large to plausibly close, and is consistent with the per-station sparsity.) What the ablation does not establish is that the aggregation has to be *geographic*; that is a separate question, taken up next.

1.12 Does the Partition Matter? Geographic vs. Random Aggregation

The ablation shows that stations must be pooled before forecasting. It is tempting to conclude that the *geographic* pooling of Section 1.3 is what creates the predictable signal — that grouping neighbouring, correlated stations is how spatial structure is exploited. That is a separate claim, and it can be tested by holding everything fixed (the 965 station series, K , the models, the protocol) and changing only how the stations are partitioned:

SKATER. The contiguous service areas used throughout this chapter.

k -means. k -means on projected site coordinates: geographic and compact, but with no contiguity constraint.

Random. The SKATER labels permuted over the stations: the same K and exactly the same cluster sizes, but no geography. Three independent permutations are drawn.

Each partition is coarsened by the rule of Section 1.3 and forecast with the graph-free GRU and with Graph WaveNet under the protocol of Section 1.6, at $K=10$ and $K=50$ (three seeds per partition; 60 training runs). As a check on the harness, the re-derived SKATER coarsening reproduces the pipeline’s stored cluster signals and coarse graph exactly. Table 1.5 and Figure 1.14 report the outcome.

Geography is not what makes the signal predictable. The random partition is not harder to forecast; it is markedly easier. At $K=10$ Graph WaveNet reaches a WAPE of 14.6% on random groups against 22.2% on the SKATER areas, and 20.8% against 34.2% at $K=50$. The targets themselves are smoother — persistence alone scores 20.3% on random groups and 29.5% on SKATER areas at $K=10$ — and the model’s skill over persistence is, if anything, higher (0.285 against 0.248). The predictability documented in Section 1.11 therefore comes from *pooling many stations*, not from pooling *neighbouring* ones. The explanation is statistical. A sum of weakly correlated bursty series averages its fluctuations out; a sum of strongly correlated

Table 1.5: Partition control (Netflix). Structure: number of contiguous clusters, mean station-to-centroid radius, mean correlation between stations of a cluster ($\bar{\rho}_{\text{in}}$) and between cluster signals ($\bar{\rho}_{\text{btw}}$). Forecast: WAPE in %, skill over persistence; medians over runs (3 seeds; 3×3 for the random partitions).

K	partition	structure				WAPE		skill	
		contiguous	radius	$\bar{\rho}_{\text{in}}$	$\bar{\rho}_{\text{btw}}$	persist.	GWN	GRU	GWN
10	SKATER	10/10	2.73	0.33	0.65	29.5	22.2	0.238	0.248
	k -means	10/10	2.58	0.33	0.66	30.1	22.8	0.233	0.244
	random	0/10	6.40	0.24	0.93	20.3	14.6	0.282	0.285
50	SKATER	50/50	1.11	0.50	0.43	44.4	34.2	0.217	0.229
	k -means	50/50	1.06	0.51	0.43	44.8	34.3	0.199	0.235
	random	0/50	6.39	0.24	0.76	27.9	20.8	0.246	0.255

ones does not. Stations in a geographic cluster are positively correlated ($\bar{\rho}_{\text{in}} = 0.33$ at $K=10$, 0.50 at $K=50$, against 0.24 for random groups; recall that adjacent stations correlate at 0.74), so a contiguous area retains local fluctuations that a city-wide sample of the same size cancels. It would therefore be wrong to say that aggregation “exploits” spatial structure: spatial correlation works *against* the smoothing that aggregation provides.

But a random partition forecasts nothing useful. The reason random groups are so smooth is that each one is a scaled-down replica of the whole city: their signals correlate at 0.93 with one another at $K=10$ (0.76 at $K=50$), against 0.65 (0.43) for the service areas (Figure 1.14(c)). Ten such forecasts carry little more than one forecast of the city total, and they do not correspond to anything a user plane function can serve: none of the random clusters is contiguous, and their members lie on average 6.4 km from the centroid, more than twice the 2.7 km of a SKATER area. The quantity the system needs is the load *of a place*, and that is intrinsically harder to predict than the load of a random sample. The difference — about 8 WAPE points at $K=10$ and 13 at $K=50$ — is the price of locality. It is a property of the problem, not a deficiency of the pipeline, and it also accounts for the rise of raw error with K in Section 1.9: smaller areas pool fewer and more strongly correlated stations.

Among geographic partitions the method matters little. k -means and SKATER produce areas of similar compactness and coherence, and their forecast quality differs by no more than 0.006 skill at either size, in opposite directions (Graph WaveNet 0.244 against 0.248 at $K=10$, 0.235 against 0.229 at $K=50$). On this site layout k -means also happened to return contiguous

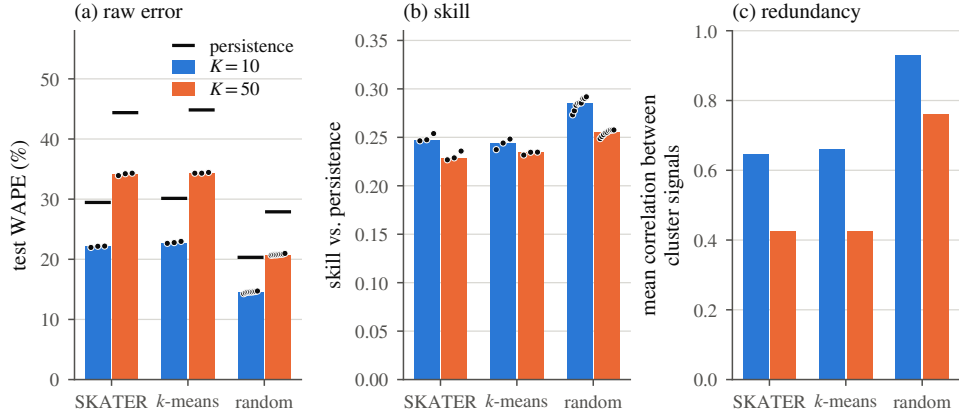


Figure 1.14: Partition control with Graph WaveNet at $K=10$ and $K=50$ (Netflix). Bars are medians and dots individual training runs. (a) Raw test WAPE, with the persistence WAPE of the same target as a black tick. (b) Skill over persistence. (c) Mean pairwise correlation between the K cluster signals. Random groupings of the same sizes are easier to forecast and slightly *more* skilful, but their signals are nearly interchangeable.

clusters in every case. The choice of SKATER therefore rests on what it guarantees by construction — contiguity, and partitions that are nested across K — and not on a forecasting advantage. Finally, the graph is only useful when it means something: Graph WaveNet improves on the graph-free GRU by about 0.01 skill on the two geographic partitions and by 0.003 on the random one at $K=10$, where the coarse graph is complete and carries no information.

Two caveats apply. The control uses one service and two values of K ; and one of the nine GRU runs on a random partition stopped after four epochs (skill 0.03), which is why medians rather than means are reported — every run is shown in Figure 1.14.

1.13 Summary

This chapter built the spatio-temporal forecaster that supplies the digital twin’s offered load. The principal points are:

1. **Scope.** The forecaster emits a dimensionless per-cluster load $\hat{d}$; physical calibration (α) and control are external. One trained model serves every capacity scenario.

2. **A documented spatial substrate.** 54 013 tiles are attributed to 965 base stations by Voronoi tessellation, giving a connected adjacency graph of 2 772 edges. On it, single-station traffic is sparse (88% of Netflix station-slots are near zero) and weakly autocorrelated, and correlation between stations is short-ranged, decaying to the city-wide background within about six hops.
3. **Contiguous, compact, unbalanced service areas.** SKATER partitions the stations into K connected areas that are nested across K and independent of the service. The areas are compact (mean radius 2.7 km at $K=10$) but far from balanced: one central cluster holds 31% of the stations and 36% of the load, a property of the city that the downstream control problem inherits.
4. **Model choice barely matters; deploy simple.** Six forecasters from linear to learned-graph span only ~ 0.06 skill; Graph WaveNet gives the best accuracy–cost trade-off and a plain GRU is near-best. The inter-cluster graph adds a small, repeatable gain ($\sim 3\%$ WAPE), and a learned adjacency (AGCRN) does not beat the Voronoi-derived one. Graph WaveNet is recommended for the twin.
5. **The advantage is uniform across clusters, grows with lead time, and is weakest at peaks.** At $K=10$ skill lies between 0.22 and 0.29 for every cluster and rises from 0.14 at 15 minutes to 0.31 at one hour; peaks are systematically under-forecast (-17% in the top load quartile), a consequence of the absolute-error objective and the main opening for improvement.
6. **The problem is well-behaved.** Skill over persistence is near-constant across graph sizes ($K = 4 \rightarrow 100$; rising raw WAPE reflects intrinsic small-cluster difficulty, not model decay) and holds across aggregate and sparse traffic types.
7. **Cluster-first is a forecasting advantage (central result).** A controlled ablation on a fixed target shows that forecasting the aggregated cluster signal directly reaches 0.21–0.23 skill, while forecasting all 965 base stations and summing is no better than persistence (≈ 0). Aggregation *denoises* sparse per-station traffic into a predictable signal.
8. **Predictability comes from pooling; geography is the requirement, and it has a price.** Random groupings of the same sizes are *easier* to forecast than the service areas (WAPE 14.6% against

22.2% at $K=10$) because they average out the local correlation that contiguous areas retain — but their signals are near-copies of the city total and map to no deployable area. Forecasting the load of a place is intrinsically harder than forecasting the load of a sample; among geographic partitions, SKATER and k -means forecast equally well, so SKATER is retained for its guaranteed contiguity.

Bibliography

- [1] Y. Li, R. Yu, C. Shahabi, Y. Liu. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. *ICLR*, 2018.
- [2] Z. Wu, S. Pan, G. Long, J. Jiang, C. Zhang. Graph WaveNet for Deep Spatial-Temporal Graph Modeling. *IJCAI*, 2019.
- [3] L. Bai, L. Yao, C. Li, X. Wang, C. Wang. Adaptive Graph Convolutional Recurrent Network for Traffic Forecasting. *NeurIPS*, 2020.
- [4] A. Zeng, M. Chen, L. Zhang, Q. Xu. Are Transformers Effective for Time Series Forecasting? *AAAI*, 2023.
- [5] R. M. Assunção, M. C. Neves, G. Câmara, C. da Costa Freitas. Efficient Regionalization Techniques for Socio-Economic Geographical Units Using Minimum Spanning Trees. *International Journal of Geographical Information Science*, 20(7):797–811, 2006.